

Computer Science Internal Assessment

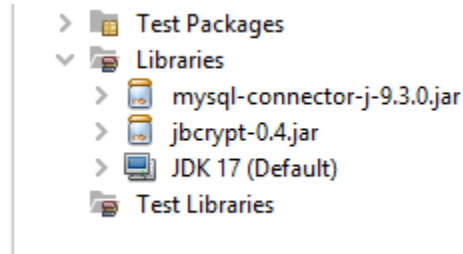
IA Project: Student Management System

Table of Contents

Imports and Dependencies.....	2
Techniques complexity used.....	2
Normal Framework.....	3
Password Encryption.....	5
Import multiple packages.....	6
Libraries.....	6
Images Package.....	6
Image handling.....	7
Static & Non-static methods.....	8
Array handling.....	10
Conditional Nested If Statements.....	11
Multiple selection statement.....	12
Complex Algorithm.....	13
Functions (parameterised, non-parameterised).....	14
The constructor definition.....	15
Class definition.....	16
CRUD Operation.....	17
Create.....	17
Read.....	18
Update.....	20
Delete.....	22
Try & Catch Block.....	24

Imports and Dependencies

In my dev journey, I have used the least of the external libraries or frameworks that exist, but I have used those that are extremely required for my Java program to function effectively(Figure 1)



No	Jar File	Purpose/Use
1	mysql-connector-j-9.3.0.jar	JDBC driver for connecting Java applications to MySQL databases.
2	jbCrypt-0.4.jar	implementation of the BCrypt password hashing function. It's commonly used to secure and verify passwords
3	JDK 17	powers backend development in full-stack apps with a modern, stable Java platform.

Techniques complexity used

- Normal Framework
- Password encryption (hash salt)
- Import multiple packages based on the requirements of the project, Images Package
- Image handling
- Static and non-static methods
- Array handling
- Conditional statement nested if
- Iteration (for loop, while loop, nested loop)
- Complex Algorithm
- Functions (parameterised, non-parameterised)
- The constructor definition
- Class definition
- Database File handling (CRUD operation)

Normal Framework

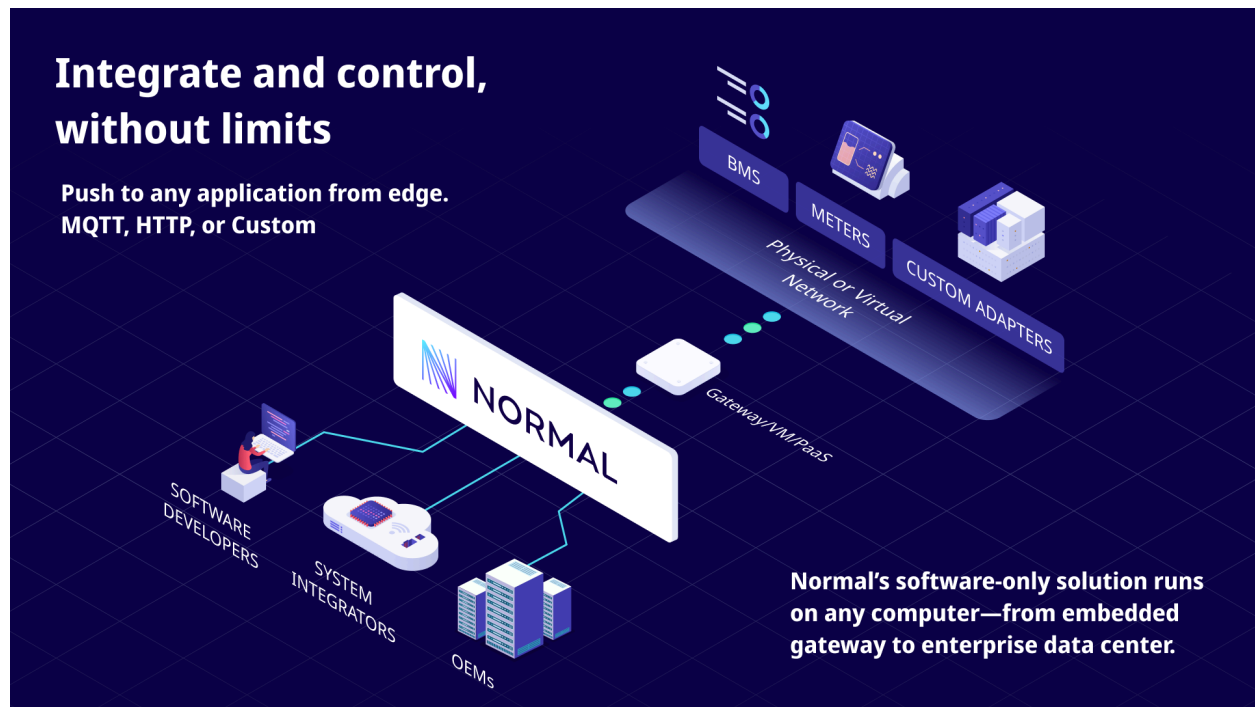
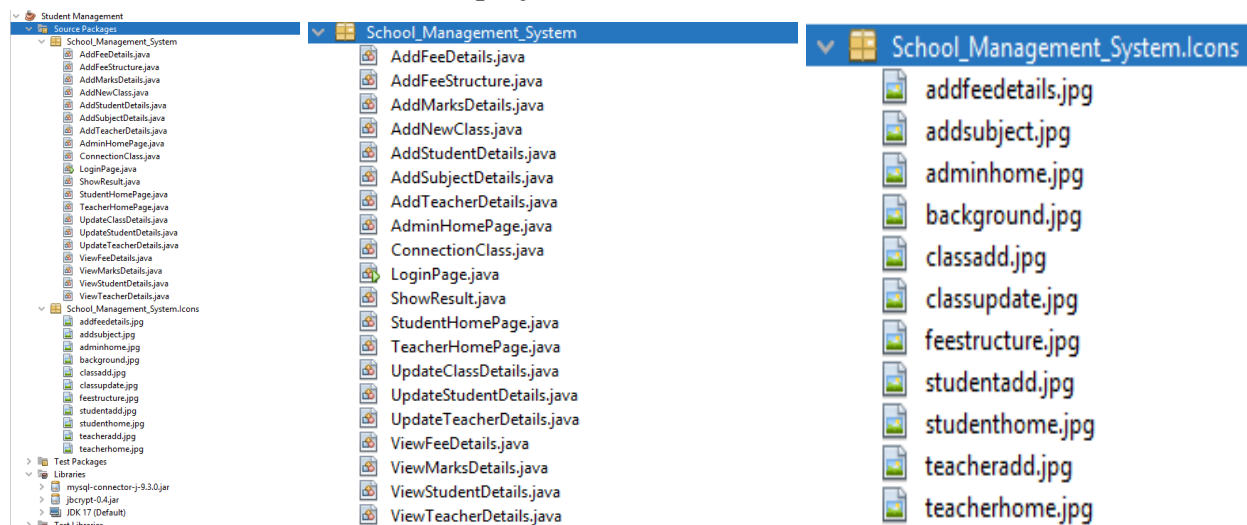


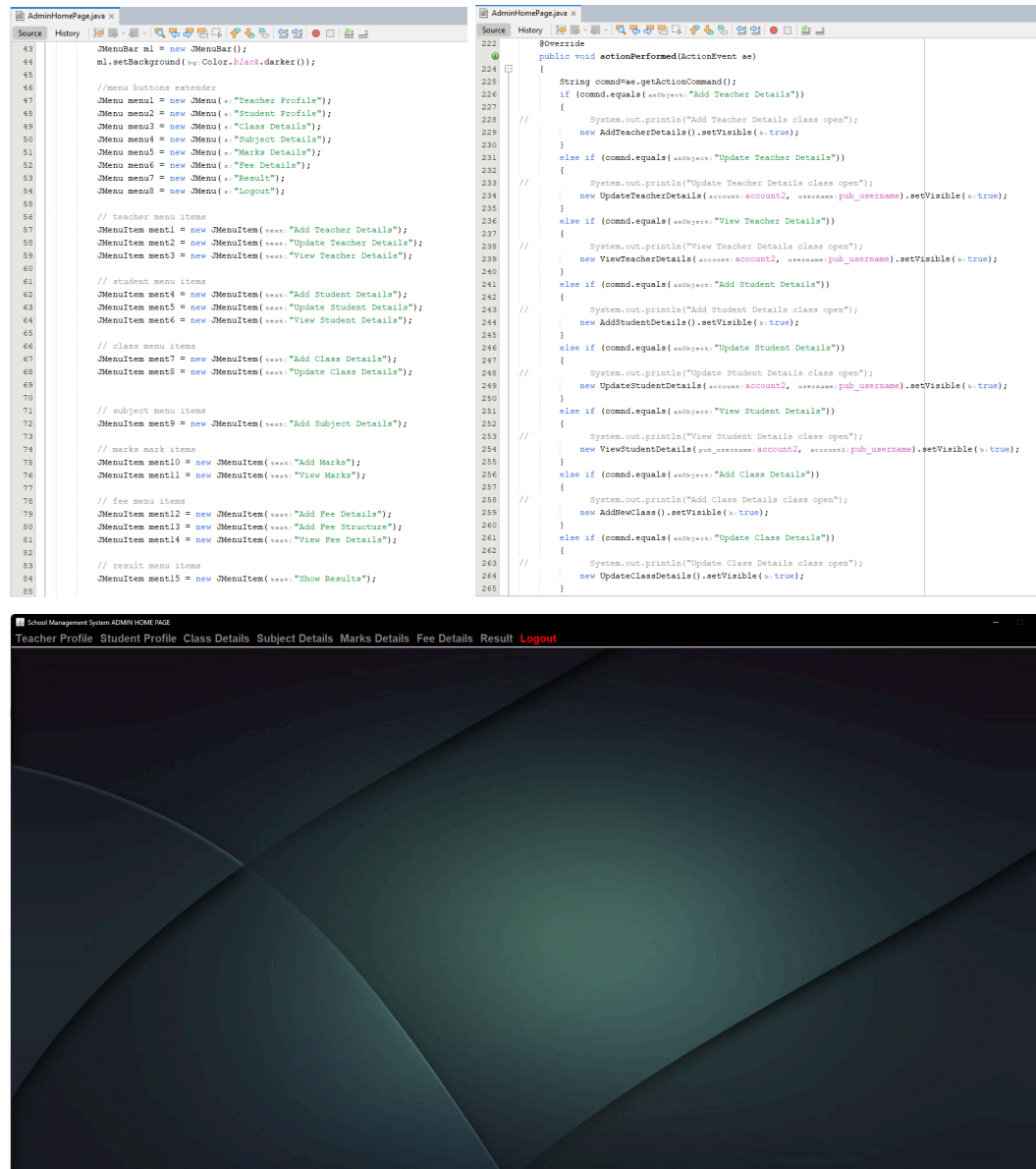
Image Credit: [Normal Framework](#)

The Normal Framework (NF) is a framework for software designed for Internet of Things (IoT) application developers, especially those working with systems. NF is built on years of experience in building, deploying, and operating cloud-connected software products. Full breakdown of the files in the project



The Project's J-Package has Visual, Model and Data Control all together compressed in every single class, with ConnectionClass.java being a file for easy instantiation of SQL Connection Objects for each Class. Images J-Package has all the Background Images used for this project

Screenshot from the AdminHomePage.java and AddSubjectDetails.java class showing the view capabilities



```

AddStudentDetails.java x
Source History
239 public void actionPerformed(ActionEvent e)
240 {
241     if (e.getSource() == b1)
242     {
243         String name = f1.getText();
244         String roll_no = f10.getText();
245         String username = f2.getText();
246         String password = f3.getText();
247         String email = f9.getText();
248         String father_name = f4.getText();
249         String phone = f5.getText();
250         String blood_group = f6.getText();
251         String gender = (String) cbl.getSelectedItem();
252         String city = f7.getText();
253         String age = f8.getText();
254         String grade = (String) cbl2.getSelectedItem();
255         String dob = f9.getText();
256         Random r = new Random();
257         String stu_id = "" + Math.abs(r.nextInt() % 100000);
258         int total_student = 0;
259
260         try
261         {
262             ConnectionClass obj = new ConnectionClass();
263             String q = "insert into student values ('" + stu_id + "','" + roll_no + "','" + name + "','" + username + "','" + password + "','" + email + "','" + father_name + "','" + phone + "','" + blood_group + "','" + city + "','" + age + "','" + gender + "','" + grade + "','" + dob + "')";
264             String q1 = "update class set enrolled = '" + roll_no + "' where class_name = '" + grade + "'";
265             String q2 = "select * from class where class_name = '" + grade + "'";
266             ResultSet rest = obj.stm.executeQuery(q2);
267             while (rest.next())
268             {
269                 total_student = Integer.parseInt(rest.getString("enrolled"));
270             }
271             if (Integer.parseInt(roll_no) > total_student)
272             {
273                 JOptionPane.showMessageDialog(null, message: "Sorry ! This class is full!");
274             }
275             else
276             {
277                 obj.stm.executeUpdate(q1);
278                 obj.stm.executeUpdate(q);
279                 JOptionPane.showMessageDialog(null, message: "Details Successfully Inserted");
280                 setVisible(false);

```

This framework helps keep all the parts together: the view, action listener and database connection for each respective section in its own file, also with the ConnectionClass.java file helps create new objects in different classes to get the connection to the database easily connected and working. This may consume a lot more storage at once but keeps everything easily known and organised for the main coder to know what and where is the code.

Password Encryption is made possible with the use of the hash salt method in PasswordUtils.java

Encryption is successfully authenticated during login, as per the screenshot.

Screenshot from addteacherdetails.java showing encryption during user creation

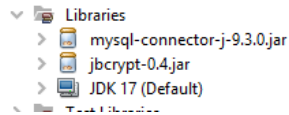
```
mysql> select * from teacher;
```

tec_id	name	username	password	email	father_name	phone	blood_group	gender	city	age	teaching_exp	dob
14774	Parth	Parth	Xal7ThMS1gmQC42kviA==	VsFntVohyCjXajexFL39QWVYvG8XQu93uotl1B0QM=	example@gmail.com	NA	1111111111	Male	Some	Some	Some	Something
59616	Dharshan	teacher1	L5VkJ2t6SP5HC3xfw6A=	AlkpuevQy8r7to1tTpv3p20m9m5H4KcTUMRvg6=	dharshan_j@gmail.com	Mr. J	1111111111	Male	Kolhapur	38	10	1/1/1995
90521	Mahi Kumari	teacher2	Vyey41gMG6e0CPTMFfCek1cB5X4T593/Aja7=	mahi_kumari@gmail.com	Mr. Kumari	2222222222	Female	Raichur	28	5	1/1/1997	

Password is encrypted during user creation for password encryption and authentication to work successfully during login and data is encrypted.

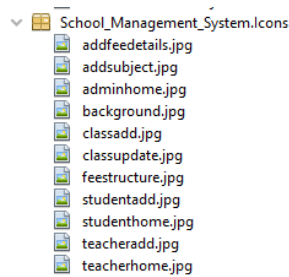
Import multiple packages

Libraries



Use of different Libraries to facilitate for the need of mysql connection through mysql-connector and jbcrypt for encryption procedures, these all have been made based on the requirements of the project.

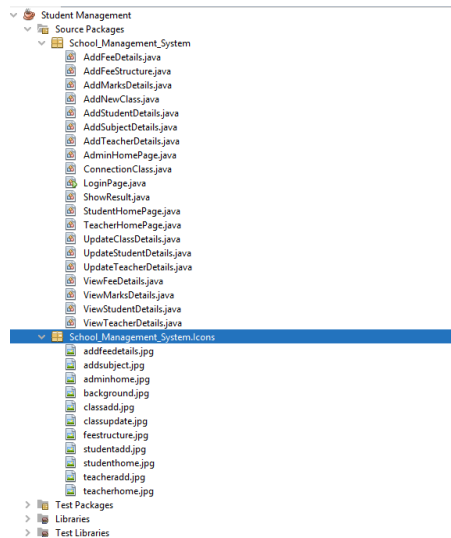
Images Package



As per the requirement of the project, the use of different images have been done through the internet by inserting them with the project files in the Images Folder/package named as School_Management_System.Icons

Image handling

Holds data of all the images for the software, mostly background images which are found from the internet



```
28 // Load background
29 ImageIcon img = new ImageIcon( location.getClass().getResource( name:"Icons/studentadd.jpg"));
30 Image scaledImg = img.getImage().getScaledInstance( width: 840, height: 600, hints: Image.SCALE_SMOOTH);
31 JLabel bgl = new JLabel(new ImageIcon( image:scaledImg));
32 bgl.setBounds( x: 0, y: 0, width: 840, height: 600);
33 bgl.setLayout( mgr: null);
34 add( comp:bgl);
```

Screenshot from AddStudentDetails, showing the Image handling using an image from the Product files, and above is the code showing the handling and mechanism of the whole process, of how the image is located and then manipulated to the requirements and used.

Static & Non-static methods

```
1 package School_Management_System;
2
3 import java.security.*;
4 import java.util.Base64;
5 public class PasswordUtils
6 {
7     // Method to hash the password with a salt
8     public static String hashPassword(String password) throws NoSuchAlgorithmException
9     {
10         // Create a salt
11         SecureRandom random = new SecureRandom();
12         byte[] salt = new byte[16];
13         random.nextBytes(salt);
14         // Hash the password with the salt
15         MessageDigest digest = MessageDigest.getInstance("SHA-256");
16         digest.update(salt); // Salt
17         byte[] hashedPassword = digest.digest(password.getBytes());
18         // Convert the hashed password to a Base64 string for storage
19         String hashedPasswordString = Base64.getEncoder().encodeToString(hashedPassword);
20         String saltString = Base64.getEncoder().encodeToString(salt);
21         // Return a combination of the salt and hashed password to store in the DB
22         return saltString + ":" + hashedPasswordString;
23     }
24 }
```

```
ViewFeeDetails.java x ConnectionClass.java x
Source History
5 public class ConnectionClass {
6     public Connection con;
7     public Statement stm;
8
9     ConnectionClass() {
10         try {
11             // Load MySQL JDBC Driver
12             Class.forName("com.mysql.cj.jdbc.Driver");
13
14             // Define connection URL
15             String url = "jdbc:mysql://localhost:330/sms";
16             String user = "root";
17             String password = ""; // used "" as no password
18
19             // Establish connection
20             con = DriverManager.getConnection(url, user, password);
21             stm = con.createStatement();
22
23             if (con != null && !con.isClosed()) {
24                 System.out.println("Connected to Database");
25             } else {
26                 System.out.println("Connection Failed");
27             }
28         } catch (ClassNotFoundException | SQLException ex) {
```

My product uses both static and non-static methods in using variables as parameters in respective classes, it follows a partial version of the Singleton Design pattern while it varies it alone does all the tasks of the connection to the database, hence it can be called as Singleton-style Database Connection Manager design.

Array handling

```

9      public class ViewFeeDetails extends JFrame {
10      {
11          String x[]={"Id","Class Name","Username","Name","Email","Total Fee","Submitted Fee","Status","Date"};
12          String y[][]=new String [21][9];
13          int i=0,j=0;
14          .....
15      }
16      .....
17      .....
18      .....
19      .....
20      .....
21      .....
22      .....
23      .....
24      .....
25      .....
26      .....
27      .....
28      .....
29      .....
30      .....
31      .....
32      .....
33      .....
34      .....
35      .....
36      .....
37      .....
38      .....
39      .....
40      .....
41      .....
42      .....
43      .....
44      .....
45      .....
46      .....
47      .....
48      .....
49      .....
50      .....
51      .....
52      .....
53      .....
54      .....
55      .....
56      .....
57      .....
58      .....
59      .....
60      .....
61      .....
62      .....
63      .....
64      .....
65      .....
66      .....
67      .....
68      .....
69      .....
70      .....
71      .....
72      .....
73      .....
74      .....
75      .....
76      .....
77      .....
78      .....
79      .....
80      .....
81      .....
82      .....
83      .....
84      .....
85      .....
86      .....
87      .....
88      .....
89      .....
90      .....
91      .....
92      .....
93      .....
94      .....
95      .....
96      .....
97      .....
98      .....
99      .....
100     .....
101     .....
102     .....
103     .....
104     .....
105     .....
106     .....
107     .....
108     .....
109     .....
110     .....
111     .....
112     .....
113     .....
114     .....
115     .....
116     .....
117     .....
118     .....
119     .....
120     .....
121     .....
122     .....
123     .....
124     .....
125     .....
126     .....
127     .....
128     .....
129     .....
130     .....
131     .....
132     .....
133     .....
134     .....
135     .....
136     .....
137     .....
138     .....
139     .....
140     .....
141     .....
142     .....
143     .....
144     .....
145     .....
146     .....
147     .....
148     .....
149     .....
150     .....
151     .....
152     .....
153     .....
154     .....
155     .....
156     .....
157     .....
158     .....
159     .....
160     .....
161     .....
162     .....
163     .....
164     .....
165     .....
166     .....
167     .....
168     .....
169     .....
170     .....
171     .....
172     .....
173     .....
174     .....
175     .....
176     .....
177     .....
178     .....
179     .....
180     .....
181     .....
182     .....
183     .....
184     .....
185     .....
186     .....
187     .....
188     .....
189     .....
190     .....
191     .....
192     .....
193     .....
194     .....
195     .....
196     .....
197     .....
198     .....
199     .....
200     .....
201     .....
202     .....
203     .....
204     .....
205     .....
206     .....
207     .....
208     .....
209     .....
210     .....
211     .....
212     .....
213     .....
214     .....
215     .....
216     .....
217     .....
218     .....
219     .....
220     .....
221     .....
222     .....
223     .....
224     .....
225     .....
226     .....
227     .....
228     .....
229     .....
230     .....
231     .....
232     .....
233     .....
234     .....
235     .....
236     .....
237     .....
238     .....
239     .....
240     .....
241     .....
242     .....
243     .....
244     .....
245     .....
246     .....
247     .....
248     .....
249     .....
250     .....
251     .....
252     .....
253     .....
254     .....
255     .....
256     .....
257     .....
258     .....
259     .....
260     .....
261     .....
262     .....
263     .....
264     .....
265     .....
266     .....
267     .....
268     .....
269     .....
270     .....
271     .....
272     .....
273     .....
274     .....
275     .....
276     .....
277     .....
278     .....
279     .....
280     .....
281     .....
282     .....
283     .....
284     .....
285     .....
286     .....
287     .....
288     .....
289     .....
290     .....
291     .....
292     .....
293     .....
294     .....
295     .....
296     .....
297     .....
298     .....
299     .....
300     .....
301     .....
302     .....
303     .....
304     .....
305     .....
306     .....
307     .....
308     .....
309     .....
310     .....
311     .....
312     .....
313     .....
314     .....
315     .....
316     .....
317     .....
318     .....
319     .....
320     .....
321     .....
322     .....
323     .....
324     .....
325     .....
326     .....
327     .....
328     .....
329     .....
330     .....
331     .....
332     .....
333     .....
334     .....
335     .....
336     .....
337     .....
338     .....
339     .....
340     .....
341     .....
342     .....
343     .....
344     .....
345     .....
346     .....
347     .....
348     .....
349     .....
350     .....
351     .....
352     .....
353     .....
354     .....
355     .....
356     .....
357     .....
358     .....
359     .....
360     .....
361     .....
362     .....
363     .....
364     .....
365     .....
366     .....
367     .....
368     .....
369     .....
370     .....
371     .....
372     .....
373     .....
374     .....
375     .....
376     .....
377     .....
378     .....
379     .....
380     .....
381     .....
382     .....
383     .....
384     .....
385     .....
386     .....
387     .....
388     .....
389     .....
390     .....
391     .....
392     .....
393     .....
394     .....
395     .....
396     .....
397     .....
398     .....
399     .....
400     .....
401     .....
402     .....
403     .....
404     .....
405     .....
406     .....
407     .....
408     .....
409     .....
410     .....
411     .....
412     .....
413     .....
414     .....
415     .....
416     .....
417     .....
418     .....
419     .....
420     .....
421     .....
422     .....
423     .....
424     .....
425     .....
426     .....
427     .....
428     .....
429     .....
430     .....
431     .....
432     .....
433     .....
434     .....
435     .....
436     .....
437     .....
438     .....
439     .....
440     .....
441     .....
442     .....
443     .....
444     .....
445     .....
446     .....
447     .....
448     .....
449     .....
450     .....
451     .....
452     .....
453     .....
454     .....
455     .....
456     .....
457     .....
458     .....
459     .....
460     .....
461     .....
462     .....
463     .....
464     .....
465     .....
466     .....
467     .....
468     .....
469     .....
470     .....
471     .....
472     .....
473     .....
474     .....
475     .....
476     .....
477     .....
478     .....
479     .....
480     .....
481     .....
482     .....
483     .....
484     .....
485     .....
486     .....
487     .....
488     .....
489     .....
490     .....
491     .....
492     .....
493     .....
494     .....
495     .....
496     .....
497     .....
498     .....
499     .....
500     .....
501     .....
502     .....
503     .....
504     .....
505     .....
506     .....
507     .....
508     .....
509     .....
510     .....
511     .....
512     .....
513     .....
514     .....
515     .....
516     .....
517     .....
518     .....
519     .....
520     .....
521     .....
522     .....
523     .....
524     .....
525     .....
526     .....
527     .....
528     .....
529     .....
530     .....
531     .....
532     .....
533     .....
534     .....
535     .....
536     .....
537     .....
538     .....
539     .....
540     .....
541     .....
542     .....
543     .....
544     .....
545     .....
546     .....
547     .....
548     .....
549     .....
550     .....
551     .....
552     .....
553     .....
554     .....
555     .....
556     .....
557     .....
558     .....
559     .....
560     .....
561     .....
562     .....
563     .....
564     .....
565     .....
566     .....
567     .....
568     .....
569     .....
570     .....
571     .....
572     .....
573     .....
574     .....
575     .....
576     .....
577     .....
578     .....
579     .....
580     .....
581     .....
582     .....
583     .....
584     .....
585     .....
586     .....
587     .....
588     .....
589     .....
590     .....
591     .....
592     .....
593     .....
594     .....
595     .....
596     .....
597     .....
598     .....
599     .....
600     .....
601     .....
602     .....
603     .....
604     .....
605     .....
606     .....
607     .....
608     .....
609     .....
610     .....
611     .....
612     .....
613     .....
614     .....
615     .....
616     .....
617     .....
618     .....
619     .....
620     .....
621     .....
622     .....
623     .....
624     .....
625     .....
626     .....
627     .....
628     .....
629     .....
630     .....
631     .....
632     .....
633     .....
634     .....
635     .....
636     .....
637     .....
638     .....
639     .....
640     .....
641     .....
642     .....
643     .....
644     .....
645     .....
646     .....
647     .....
648     .....
649     .....
650     .....
651     .....
652     .....
653     .....
654     .....
655     .....
656     .....
657     .....
658     .....
659     .....
660     .....
661     .....
662     .....
663     .....
664     .....
665     .....
666     .....
667     .....
668     .....
669     .....
670     .....
671     .....
672     .....
673     .....
674     .....
675     .....
676     .....
677     .....
678     .....
679     .....
680     .....
681     .....
682     .....
683     .....
684     .....
685     .....
686     .....
687     .....
688     .....
689     .....
690     .....
691     .....
692     .....
693     .....
694     .....
695     .....
696     .....
697     .....
6
```

[illegible]

Array is handled in View Classes, the above is a snippet from ViewFeeDetails, showing how the array is used for representing data from the database, and then the data is individually implemented to showcase a database type of data in the product.

Conditional Nested If Statements

Nested If Statement

During the product development phase, I have used nested if statements to open the correct panel based on what is called to open the correct menu of the project based on the command called and what needs to be run and opened given in that code. This also shows a variation of polymorphism.

```
222 8Override
223 0
224 public void actionPerformed(ActionEvent ae)
225 {
226     String cmd=ae.getActionCommand();
227     if (cmd.equals("Add Teacher Details"))
228     {
229         System.out.println("Add Teacher Details class open");
230         new AddTeacherDetails().setVisible(true);
231     }
232     else if (cmd.equals("Update Teacher Details"))
233     {
234         System.out.println("Update Teacher Details class open");
235         new UpdateTeacherDetails(account:account2, username:pub_username).setVisible(true);
236     }
237     else if (cmd.equals("View Teacher Details"))
238     {
239         System.out.println("View Teacher Details class open");
240         new ViewTeacherDetails(account:account2, username:pub_username).setVisible(true);
241     }
242     else if (cmd.equals("Add Student Details"))
243     {
244         System.out.println("Add Student Details class open");
245         new AddStudentDetails().setVisible(true);
246     }
247     else if (cmd.equals("Update Student Details"))
248     {
249         System.out.println("Update Student Details class open");
250         new UpdateStudentDetails(account:account2, username:pub_username).setVisible(true);
251     }
252     else if (cmd.equals("View Student Details"))
253     {
254         System.out.println("View Student Details class open");
255         new ViewStudentDetails(pub_username, account2, account:pub_username).setVisible(true);
256     }
257     else if (cmd.equals("Add Class Details"))
258     {
259         System.out.println("Add Class Details class open");
260         new AddNewClass().setVisible(true);
261     }
262     else if (cmd.equals("Update Class Details"))
263     {
264         System.out.println("Update Class Details class open");
265         new UpdateClassDetails().setVisible(true);
266     }
267     else if (cmd.equals("Add Subject Details"))
268     {
269         System.out.println("Add Subject Details class open");
270         new AddSubjectDetails().setVisible(true);
271     }
272     else if (cmd.equals("Add Marks"))
273     {
274         System.out.println("Add Mark Subject class open");
275         new AddMarksDetails().setVisible(true);
276     }
277     else if (cmd.equals("View Marks"))
278     {
279         System.out.println("Add Mark Subject class open");
280         new ViewMarksDetails(pub_username, account2).setVisible(true);
281     }
282     else if (cmd.equals("Add Fee Details"))
283     {
284         System.out.println("Add Fee Details class open");
285         new AddFeeDetails().setVisible(true);
286     }
287     else if (cmd.equals("Add Fee Structure"))
288     {
289         System.out.println("Add Fee Structure class open");
290         new AddFeeStructure().setVisible(true);
291     }
292     else if (cmd.equals("View Fee Details"))
293     {
294         System.out.println("View Fee Details class open");
295         new ViewFeeDetails(pub_username, account2).setVisible(true);
296     }
297     else if (cmd.equals("Show Results"))
298     {
299         System.out.println("Show Results class open");
300         new ShowResult(pub_username, account2).setVisible(true);
301     }
302     else if (cmd.equals("Exit"))
303     {
304         System.out.println("You are Logged Out");
305         this.setVisible(false);
306         new LoginPage();
307     }
```

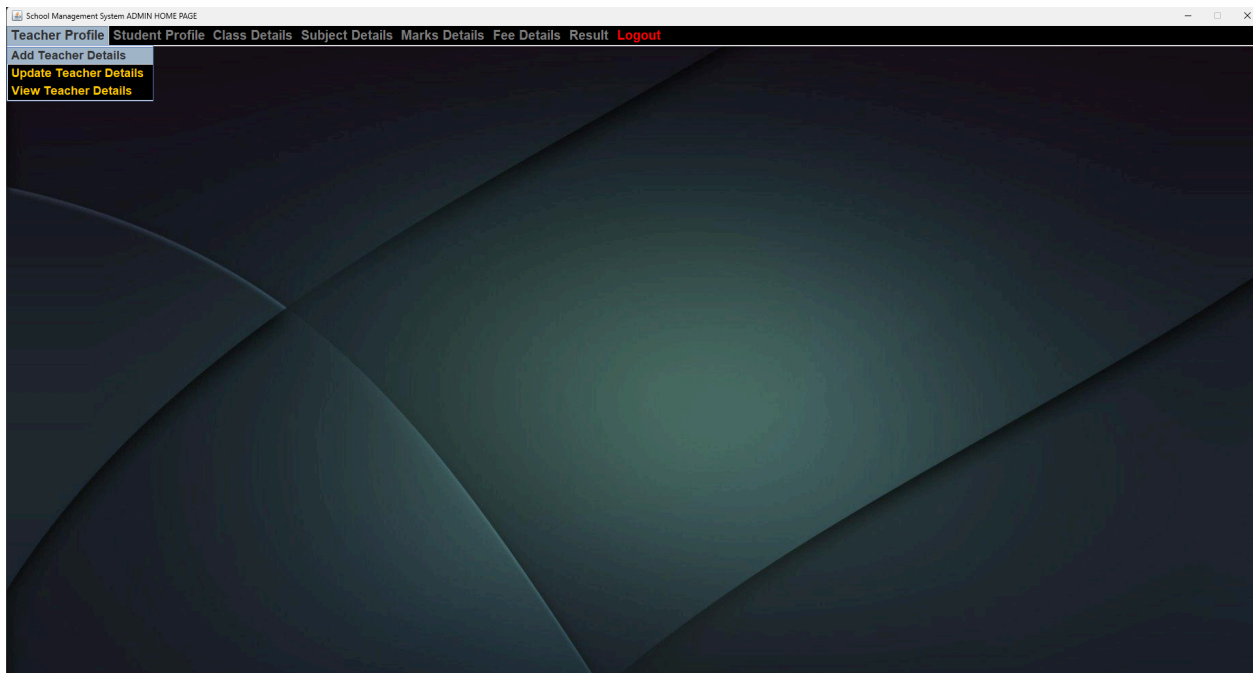


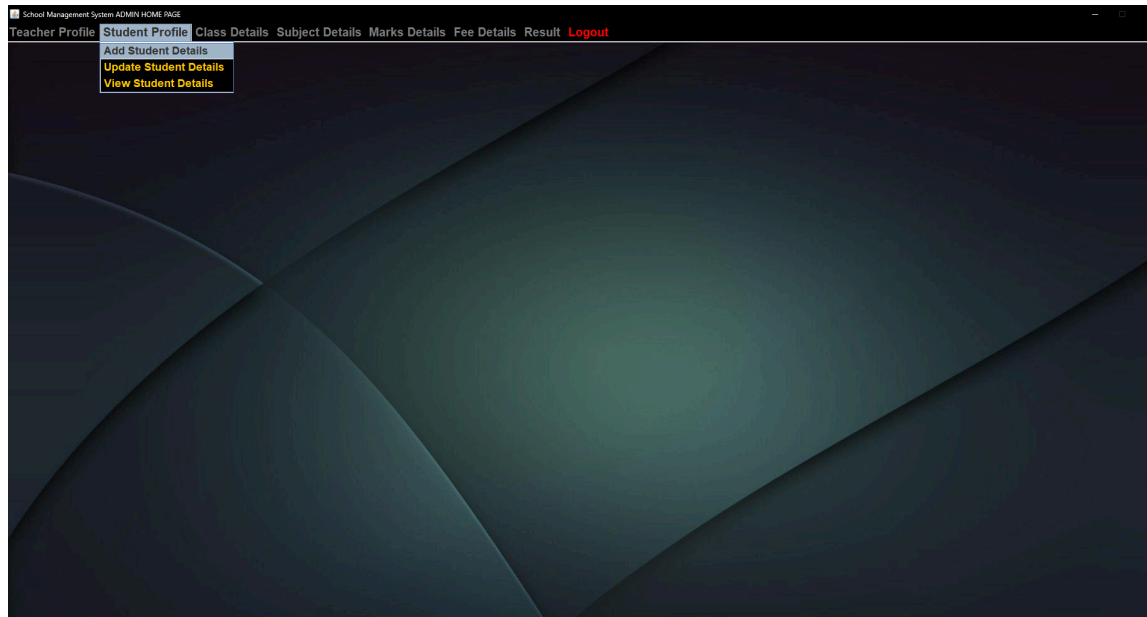
Image: Admin view of the Menu Panels available to view

Multiple selection statement

```

222 @Override
223 public void actionPerformed(ActionEvent ae)
224 {
225     String cmd=ae.getActionCommand();
226     if (cmd.equals(ae.getSource().getText()))
227     {
228         System.out.println("Add Teacher Details class open");
229         new AddTeacherDetails().setVisible(true);
230     }
231     else if (cmd.equals(ae.getSource().getText()))
232     {
233         System.out.println("Update Teacher Details class open");
234         new UpdateTeacherDetails(account2, username(pub_username)).setVisible(true);
235     }
236     else if (cmd.equals(ae.getSource().getText()))
237     {
238         System.out.println("View Teacher Details class open");
239         new ViewTeacherDetails(account2, username(pub_username)).setVisible(true);
240     }
241     else if (cmd.equals(ae.getSource().getText()))
242     {
243         System.out.println("Add Student Details class open");
244         new AddStudentDetails().setVisible(true);
245     }
246     else if (cmd.equals(ae.getSource().getText()))
247     {
248         System.out.println("Update Student Details class open");
249         new UpdateStudentDetails(account2, username(pub_username)).setVisible(true);
250     }
251     else if (cmd.equals(ae.getSource().getText()))
252     {
253         System.out.println("View Student Details class open");
254         new ViewStudentDetails(pub_username, account2, username(pub_username)).setVisible(true);
255     }
256     else if (cmd.equals(ae.getSource().getText()))
257     {
258         System.out.println("Add Class Details class open");
259         new AddNewClass().setVisible(true);
260     }
261     else if (cmd.equals(ae.getSource().getText()))
262     {
263         System.out.println("Update Class Details class open");
264         new UpdateClassDetails().setVisible(true);
265     }
266     else if (cmd.equals(ae.getSource().getText()))
267     {
268         System.out.println("Add Subject Details class open");
269         new AddSubjectDetails().setVisible(true);
270     }
271     else if (cmd.equals(ae.getSource().getText()))
272     {
273         System.out.println("Add Mark Subject class open");
274         new AddMarksDetails().setVisible(true);
275     }
276     else if (cmd.equals(ae.getSource().getText()))
277     {
278         System.out.println("Add Mark Subject class open");
279         new ViewMarksDetails(pub_username, account2).setVisible(true);
280     }
281     else if (cmd.equals(ae.getSource().getText()))
282     {
283         System.out.println("Add Fee Details class open");
284         new AddFeeDetails().setVisible(true);
285     }
286     else if (cmd.equals(ae.getSource().getText()))
287     {
288         System.out.println("Add Fee Structure class open");
289         new AddFeeStructure().setVisible(true);
290     }
291     else if (cmd.equals(ae.getSource().getText()))
292     {
293         System.out.println("View Fee Details class open");
294         new ViewFeeDetails(pub_username, account2).setVisible(true);
295     }
296     else if (cmd.equals(ae.getSource().getText()))
297     {
298         System.out.println("Show Results class open");
299         new ShowResult(pub_username, account2).setVisible(true);
300     }
301     else if (cmd.equals(ae.getSource().getText()))
302     {
303         System.out.println("You are Logged Out");
304         this.setVisible(false);
305         new LoginPage();
306     }
307 }

```



Nested loops are used to work on in the process of iteration helping in repeatedly execution of a block of code until a certain condition is met or for a fixed number of times.

```

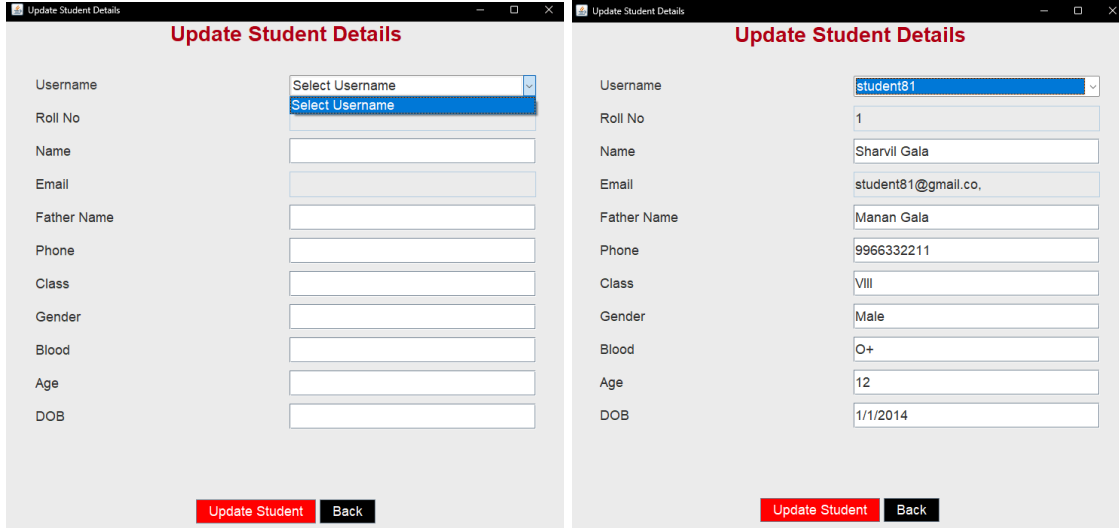
182 // ch02_addItemListener fow ItemListener()
183
184 @Override
185 public void itemStateChanged(ItemEvent e)
186 {
187     if (e.getStateChange() == ItemEvent.SELECTED)
188     {
189         String username = obj.getSelectedItemId();
190         String class_name = obj.getSelectedItemId();
191         try {
192             ConnectionClass obj = new ConnectionClass();
193             // Get student name and email
194             String q = "SELECT name, email FROM student WHERE username=" + username + " ";
195             ResultSet rs = obj.stm.executeQuery(q);
196
197             if (rs.next()) {
198                 (rs.getString("name"));
199                 (rs.getString("email"));
200             }
201             // Get full fee from fee structure
202             String qfee = "SELECT fee_rs FROM fee_structure WHERE class_name=" + class_name + " ";
203             ResultSet rsFee = obj.stm.executeQuery(qfee);
204
205             if (rsFee.next())
206             {
207                 int fullFee = Integer.parseInt(rsFee.getString("fee_rs"));
208                 int paid = 0;
209                 // Get total paid by student
210                 String qpaid = "SELECT SUM(CAST(subtotal fee AS DECIMAL)) AS total_paid FROM student_fee WHERE username=" + username + " ";
211                 ResultSet rsPaid = obj.stm.executeQuery(qpaid);
212                 if (rsPaid.next()) as rsPaid.getString("total_paid") != null)
213                 {
214                     paid = Integer.parseInt(rsPaid.getString("total_paid"));
215
216                     int remaining = fullFee - paid;
217                     if (remaining < 0)
218                         remaining = 0;
219                     (rs.getString("remaining"));
220                 }
221             }
222         } catch (Exception ex) {
223             ex.printStackTrace();
224         }
225     }
226 }
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

In complex algorithms, I am using my AddFeeDetails, fee calculation method which calculates the pending amount of fees when selected grade and username in Add Fee Details, UI it will calculate the total fee as per the fee structure minused fee data which have been inserted as completed and those which have been inserted as due will be added upon, whilst not adding past the fee structure limit for based on the student's class specification. Along with during data entry into the system the algorithm does not allow fees larger then fee structure to be added, nor accept the full amount of paid fees to be marked as due.

Functions (parameterised, non-parameterised)

```
245     else if (comnd.equals(anObject:"Update Student Details"))
246     {
247         //      System.out.println("Update Student Details class open");
248         new UpdateStudentDetails(account:account2, username:pub_username).setVisible(b:true);
249     }
250     else if (comnd.equals(anObject:"View Student Details"))
251     {
252         //      System.out.println("View Student Details class open");
253         new ViewStudentDetails(pub_username:account2, account2:pub_username).setVisible(b:true);
254     }
255     else if (comnd.equals(anObject:"Add Class Details"))
256     {
257         //      System.out.println("Add Class Details class open");
258         new AddNewClass().setVisible(b:true);
259     }
```



For functions, from AdminHomePage, the parameters when opening certain System parts which have critical and points through which unauthorised access can be created such as Update Student Details, where it is blocked by parameters As shown in Image 1 of the panel showing unaccessibility for data safety and integrity and Image 2 is showing proper functionality when the parameters are acheived. While Class details which condole none to low harm do not have parameters and depict non parameterised functions.

The constructor definition

```
7
8 public class UpdateStudentDetails extends JFrame implements ActionListener
9 {
10     JLabel l1, l2, l3, l4, l5, l6, l7, l8, l9, l10, l11, l12, l13, l14;
11     JPanel p1, p2, p3;
12     JTextField tf1, tf2, tf3, tf4, tf5, tf6, tf7, tf8, tf9, tf10, tf11;
13     Choice chl;
14     JButton bt1, bt2;
15     public String pubu, account2;
16
17     UpdateStudentDetails(String account, String username)
18     {
19         super(title: "Update Student Details");
20         setSize( width: 760, height: 720);
21         setLocation( x: 550, y: 200);
22         setVisible( b: true);
23         account2=account;
24         pubu=username;
25
26         Font f = new Font( name: "Arial", style: Font.BOLD, size: 28);
27         Font fl = new Font( name: "Arial", style: Font.PLAIN, size: 18);
28
29         chl=new Choice();
30         chl.setFont( f: fl);
31         chl.add( item: "Select Username");
32
33     }
34
35     else if (comnd.equals( anObject: "Update Student Details"))
36     {
37         // System.out.println("Update Student Details class open");
38         new UpdateStudentDetails( account: account2, username: pub_username).setVisible( b: true);
39     }
40 }
```

In this constructor, UpdateStudentDetails, is used to initialise all the data members and update the data in the database with the new data value.

Class definition

```

9      public class AddMarksDetails extends JFrame implements ActionListener
10     {
11         JLabel j1, j2, j3, j4, j5, j6, j7, j8, j9, j10, j11, j12, j13, j14, j15, j16, j17, j18, j19, j20, j21, j22, j23, j24, j25, j26, j27, j28, j29, j30, j31, j32, j33, j34, j35, j36, j37, j38, j39, j40, j41, j42, j43, j44, j45, j46, j47, j48, j49, j50, j51, j52, j53, j54, j55, j56, j57, j58, j59, j60, j61, j62, j63, j64, j65, j66, j67, j68, j69, j70, j71, j72, j73, j74, j75, j76, j77, j78, j79, j80, j81, j82, j83, j84, j85, j86, j87, j88, j89, j90, j91, j92, j93, j94, j95, j96, j97, j98, j99, j100, j101, j102, j103, j104, j105, j106, j107, j108, j109, j110, j111, j112, j113, j114, j115, j116, j117, j118, j119, j120, j121, j122, j123, j124, j125, j126, j127, j128, j129, j130, j131, j132, j133, j134, j135, j136, j137, j138, j139, j140, j141, j142, j143, j144, j145, j146, j147, j148, j149, j150, j151, j152, j153, j154, j155, j156, j157, j158, j159, j160, j161, j162, j163, j164, j165, j166, j167, j168, j169, j170, j171, j172, j173, j174, j175, j176, j177, j178, j179, j180, j181, j182, j183, j184, j185, j186, j187, j188, j189, j190, j191, j192, j193, j194, j195, j196, j197, j198, j199, j200, j201, j202, j203, j204, j205, j206, j207, j208, j209, j210, j211, j212, j213, j214, j215, j216, j217, j218, j219, j220, j221, j222, j223, j224, j225, j226, j227, j228, j229, j230, j231, j232, j233, j234, j235, j236, j237, j238, j239, j240, j241, j242, j243, j244, j245, j246, j247, j248, j249, j250, j251, j252, j253, j254, j255, j256, j257, j258, j259, j260, j261, j262, j263, j264, j265, j266, j267, j268, j269, j270, j271, j272, j273, j274, j275, j276, j277, j278, j279, j280, j281, j282, j283, j284, j285, j286, j287, j288, j289, j290, j291, j292, j293, j294, j295, j296, j297, j298, j299, j300, j301, j302, j303, j304, j305, j306, j307, j308, j309, j310, j311, j312, j313, j314, j315, j316, j317, j318, j319, j320, j321, j322, j323, j324, j325, j326, j327, j328, j329, j330, j331, j332, j333, j334, j335, j336, j337, j338, j339, j340, j341, j342, j343, j344, j345, j346, j347, j348, j349, j350, j351, j352, j353, j354, j355, j356, j357, j358, j359, j360, j361, j362, j363, j364, j365, j366, j367, j368, j369, j370, j371, j372, j373, j374, j375, j376, j377, j378, j379, j380, j381, j382, j383, j384, j385, j386, j387, j388, j389, j390, j391, j392, j393, j394, j395, j396, j397, j398, j399, j400, j401, j402, j403, j404, j405, j406, j407, j408, j409, j410, j411, j412, j413, j414, j415, j416, j417, j418, j419, j420, j421, j422, j423, j424, j425, j426, j427, j428, j429, j430, j431, j432, j433, j434, j435, j436, j437, j438, j439, j440, j441, j442, j443, j444, j445, j446, j447, j448, j449, j450, j451, j452, j453, j454, j455, j456, j457, j458, j459, j460, j461, j462, j463, j464, j465, j466, j467, j468, j469, j470, j471, j472, j473, j474, j475, j476, j477, j478, j479, j480, j481, j482, j483, j484, j485, j486, j487, j488, j489, j490, j491, j492, j493, j494, j495, j496, j497, j498, j499, j500, j501, j502, j503, j504, j505, j506, j507, j508, j509, j510, j511, j512, j513, j514, j515, j516, j517, j518, j519, j520, j521, j522, j523, j524, j525, j526, j527, j528, j529, j530, j531, j532, j533, j534, j535, j536, j537, j538, j539, j540, j541, j542, j543, j544, j545, j546, j547, j548, j549, j550, j551, j552, j553, j554, j555, j556, j557, j558, j559, j560, j561, j562, j563, j564, j565, j566, j567, j568, j569, j570, j571, j572, j573, j574, j575, j576, j577, j578, j579, j580, j581, j582, j583, j584, j585, j586, j587, j588, j589, j590, j591, j592, j593, j594, j595, j596, j597, j598, j599, j600, j601, j602, j603, j604, j605, j606, j607, j608, j609, j610, j611, j612, j613, j614, j615, j616, j617, j618, j619, j620, j621, j622, j623, j624, j625, j626, j627, j628, j629, j630, j631, j632, j633, j634, j635, j636, j637, j638, j639, j640, j641, j642, j643, j644, j645, j646, j647, j648, j649, j650, j651, j652, j653, j654, j655, j656, j657, j658, j659, j660, j661, j662, j663, j664, j665, j666, j667, j668, j669, j670, j671, j672, j673, j674, j675, j676, j677, j678, j679, j680, j681, j682, j683, j684, j685, j686, j687, j688, j689, j690, j691, j692, j693, j694, j695, j696, j697, j698, j699, j700, j701, j702, j703, j704, j705, j706, j707, j708, j709, j710, j711, j712, j713, j714, j715, j716, j717, j718, j719, j720, j721, j722, j723, j724, j725, j726, j727, j728, j729, j730, j731, j732, j733, j734, j735, j736, j737, j738, j739, j740, j741, j742, j743, j744, j745, j746, j747, j748, j749, j750, j751, j752, j753, j754, j755, j756, j757, j758, j759, j760, j761, j762, j763, j764, j765, j766, j767, j768, j769, j770, j771, j772, j773, j774, j775, j776, j777, j778, j779, j780, j781, j782, j783, j784, j785, j786, j787, j788, j789, j790, j791, j792, j793, j794, j795, j796, j797, j798, j799, j800, j801, j802, j803, j804, j805, j806, j807, j808, j809, j810, j811, j812, j813, j814, j815, j816, j817, j818, j819, j820, j821, j822, j823, j824, j825, j826, j827, j828, j829, j830, j83
```

In my project, the class acts as , AddMarksDetails and ViewMarksDetails, which extend JFrame to create GUI windows in a Student Management System. AddMarksDetails is used to input and store student marks, while ViewMarksDetails is used to retrieve and display marks in a JTable from the database.

CRUD Operation

Create

This CRUD operation in my product is a minimal requirement of the client. Each of the components (User, student, teacher, subject etc) have CRUD operation. The CRUD operation uses the Front-End(NetBeans IDE -swing components) and Back-End(SQL Database) mechanism to store data efficiently .

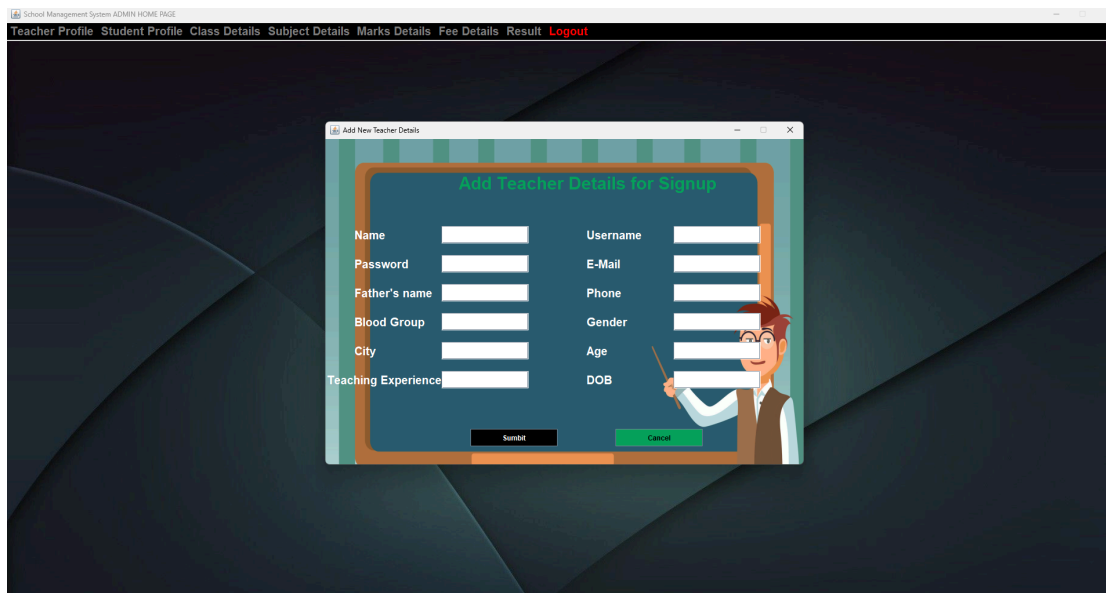


Image: Admin view of Teacher profile creation

```
187 @Override
188 public void actionPerformed(ActionEvent ae)
189 {
190     if(ae.getSource() != null)
191     {
192         String name = f1.getText();
193         String username = f2.getText();
194         String password = f3.getText();
195         // Hash the password with a salt
196         String hashedPassword = null;
197         try {
198             hashedPassword = PasswordUtils.hashPassword(password);
199         }
200         catch (NoSuchAlgorithmException ex) { }
201         //Logger.getLogger(UserScreen.class.getName()).log(Level.SEVERE, null, ex);
202         String email = f4.getText();
203         String father_name = f5.getText();
204         String phone = f6.getText();
205         String blood_group = f7.getText();
206         String gender = f8.getText();
207         String city = f9.getText();
208         String age = f10.getText();
209         String teaching_exp = f11.getText();
210         String dob = f12.getText();
211         Random r = new Random();
212         String tec_id = "Math.abs(r.nextInt() * 100000);
213
214         try
215         {
216             ConnectionClass obj = new ConnectionClass();
217             String q = "insert into teacher values('"+tec_id+"','"+name+"','"+username+"','"+hashedPassword+"','"+email+"','"+father_name
218             obj.stm.executeUpdate(q);
219             JOptionPane.showMessageDialog(parentComponent, null, message: "Details Successfully Inserted");
220             setVisible(false);
221         }
222         catch (Exception ex)
223         {
224             ex.printStackTrace();
225         }
226     }
227 }
```

Code of TeacherProfile.java

```
mysql> select * from teacher;
```

tec_id	name	username	password	email	father_name	phone	blood_group	gender	city	age	teaching_exp	dob
59016	Dharshan	teacher1	LJK2Tb6SPSHccg3xfw6A==:ALkpeuYqf8TLoEttPtVu3BP2oDyNSHMKgTUNRvGE=	dharshan.j@gmail.com	Mr. J	1111111111	O+	Male	Kolhapur	31	10	1/1/1995
90921	Rahi Kumar	teacher2	5/dLj08rjvJf5/ygRXSP1w==:Yehya1tgCH6Yoc8P7MPfncE1kcBtSX0NtR593/AjaT=	nahi.kumar@gmail.com	Mr. Kumar	2222222222	O+	Female	Raichur	28	5	1/1/1997

2 rows in set (0.01 sec)

Record in Database viewed from MySQL 8.0 Command Line Client (hashing is enabled, depict it's working during CRUD function's **CREATE** to be visible for the following screenshots)

Read

My projects also use the CRUD's Read Function in a way to view the details that already exist in the database

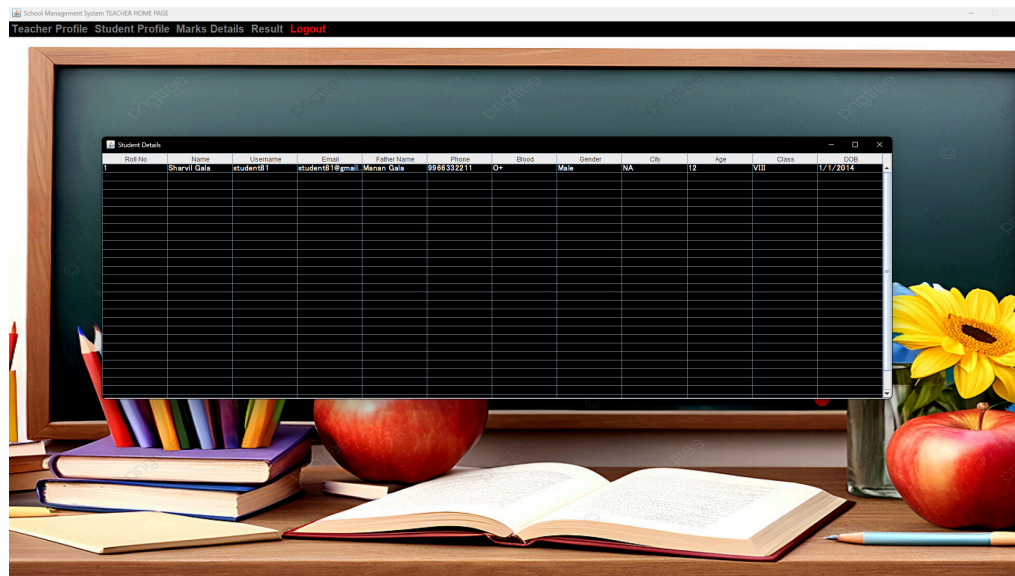


Image: Teacher view of ViewStudentDetails

```
18 ViewStudentDetails(String pub_username, String account2)
19 {
20     super("Student Details");
21     setSize(1500, 400);
22     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23     setLocation(1, 1);
24     f=new Font("MS UI Gothic", style:Font.BOLD, size:15);
25
26     pu=pub_username;
27     pa=account2;
28
29     System.out.println(pu + "pub_username" + "Account type : " + pa);
30     try
31     {
32         ConnectionClass obj=new ConnectionClass();
33         if(pa.equals("Student"))
34         {
35             query="select * from student where username='"+pu+"'";
36         }
37         else if(pa.equals("Admin"))
38         {
39             query="select * from student";
40         }
41         else if(pa.equals("Teacher"))
42         {
43             query="select * from student";
44         }
45         ResultSet rest=obj.stm.executeQuery(query);
46         while(rest.next())
47         {
48             y[i][j]=rest.getString("roll_no");
49             y[i][j]=rest.getString("name");
50             y[i][j]=rest.getString("username");
51             y[i][j]=rest.getString("email");
52             y[i][j]=rest.getString("father_name");
53             y[i][j]=rest.getString("phone");
54             y[i][j]=rest.getString("blood");
55             y[i][j]=rest.getString("gender");
56             y[i][j]=rest.getString("city");
57             y[i][j]=rest.getString("age");
58             y[i][j]=rest.getString("class");
59             y[i][j]=rest.getString("dob");
60             i++;
61             j=0;
62         }
63     }
64 }
```

Code from ViewStudentDetails.java

```
mysql> select * from student;
```

stu_id	roll_no	name	username	password	email	father_name	phone	blood	gender	city	age	class	dob
1722	1	Deshna Jain	student1	student123	deshna.jain@sgschool.in	Na	999999999	O+	Female	Kolhapur	16	IX	1/1/2008
41647	1	Sukran Nirankari	student2	student123	sukran.nirankari@sgschool.in	NA	999999999	O+	Male	Kolhapur	16	IX	1/1/2008
98342	2	Rishidatta	student3	student123	rishidatta.shah@sgschool.in	NA	666666666	O+	Male	Nipani	16	VIII	1/1/2008

3 rows in set (0.00 sec)

Record in Database viewed from MySQL 8.0 Command Line Client (hashing was disabled for CRUD function's **READ** to be visible for following screenshots)

Update

The project also uses CRUD's Update function to update details from within the program, which updates in the database in real time. Which also gets changed in the database as per what is needed.

School Management System ADMIN HOME PAGE

Teacher Profile Student Profile Class Details Subject Details Marks Details Fee Details Result Logout

Update Teacher Details

Username: teacher1

Name: Veejay

Email: teacher@gmail.com

Father Name: Beejay

Phone: 888888888

City: Kolhapur

Gender: Male

Blood: O+

Age: 40

DOB: 1/1/1985

Experience: 15

Update Teacher Back

Image: Admin view of Update Teacher Details, before Updating in program

```
mysql> select * from teacher;
```

tec_id	name	username	password	email	father_name	phone	blood_group	gender	city	age	teaching_exp	dob
22695	Veejay	teacher1	teacher123	teacher@gmail.com	Beejay	888888888	O+	Male	Kolhapur	40	15	1/1/1985
75983	Santanu Ghosh	teacher2	teacher123	santanu.ghosh@sgschool.in	NA	9999999999	O+	Male	Kolkata	39	15	1/1/1986

2 rows in set (0.00 sec)

Record in Database viewed from MySQL 8.0 Command Line Client, before update

School Management System ADMIN HOME PAGE

Teacher Profile Student Profile Class Details Subject Details Marks Details Fee Details Result Logout

Update Teacher Details

Username: teacher1

Name: Veejay

Email: teacher@gmail.com

Father Name: Beejay

Phone: 888888888

City: Kolhapur

Gender: Male

Blood: O+

Age: 41

DOB: 1/1/1985

Experience: 15

Update Teacher Back

Image: Admin view of Update Teacher Details, after Updating in program (change age +1)

```
mysql> select * from teacher;
```

tec_id	name	username	password	email	father_name	phone	blood_group	gender	city	age	teaching_exp	dob
22695	Veejay	teacher1	teacher123	teacher@gmail.com	Beejay	888888888	O+	Male	Kolhapur	41	15	1/1/1985
75983	Santanu Ghosh	teacher2	teacher123	santanu.ghosh@sgischool.in	NA	99999999999	O+	Male	Kolkata	39	15	1/1/1986

2 rows in set (0.00 sec)

Record in Database viewed from MySQL 8.0 Command Line Client, after update

```

198
199
200 @Override
201 public void actionPerformed(ActionEvent ae)
202 {
203     if(ae.getSource() == bt1)
204     {
205         String username=chl.getSelectedItemAt();
206         String name=tf1.getText();
207         String email=tf2.getText();
208         String father_name=tf3.getText();
209         String phone=tf4.getText();
210         String city=tf5.getText();
211         String gender=tf6.getText();
212         String blood_group=tf7.getText();
213         String age=tf8.getText();
214         String dob=tf9.getText();
215         String exp=tf10.getText();
216
217         try
218         {
219             ConnectionClass obj=new ConnectionClass();
220             String q = "update teacher set name='"+name+"', email='"+email+"', father_name='"+father_name+"', phone='"+phone+"', city='"+city+"', gender='"+gender+"', blood_g
221             obj.stm.executeUpdate( string: q);
222             int result = obj.stm.executeUpdate( string: q);
223             if (result == 1)
224             {
225                 JOptionPane.showMessageDialog( parentComponent: null, message: "Class Details Successfully Updated");
226                 this.setVisible( b: false);
227             }
228             else
229             {
230                 JOptionPane.showMessageDialog( parentComponent: null, message: "Please Fill all the Details");
231                 this.setVisible( b: false);
232                 new UpdateClassDetails().setVisible( b: true);
233             }
234             setVisible( b: false);
235         }
236         catch(Exception ex)
237         {
238             ex.printStackTrace();
239         }
240     }
241     if(ae.getSource() == bt2)
242     {

```

Code from UpdateTeacherDetails.java

Delete

This function helps remove certain data from the database from within the application and works with instant changes

For this I have implemented it into the ViewTeacherDetails, for which I did not use it in the Read using Admin Account, instead used a Teacher Account with View Student Details as View Teacher Details has parameters to hide other teachers Details and Show only that accounts related details registered into the system.

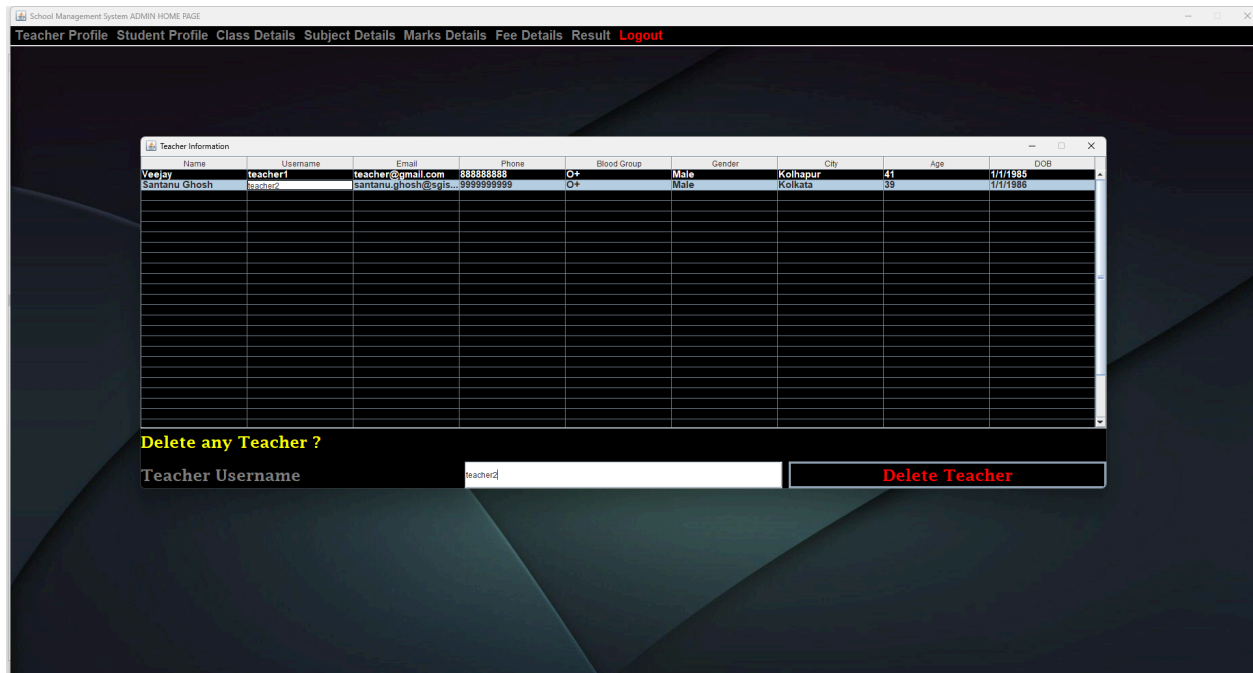


Image: Admin view of ViewTeacherDetails, Before Delete

```
mysql> select * from teacher;
```

tec_id	name	username	password	email	father_name	phone	blood_group	gender	city	age	teaching_exp	dob
22695	Veejay	teacher1	teacher123	teacher@gmail.com	Beejay	888888888	O+	Male	Kolhapur	41	15	1/1/1985
75983	Santanu Ghosh	teacher2	teacher123	santanu.ghosh@sgischool.in	NA	999999999	O+	Male	Kolkata	39	15	1/1/1986

2 rows in set (0.00 sec)

Record in Database viewed from MySQL 8.0 Command Line Client, before Delete

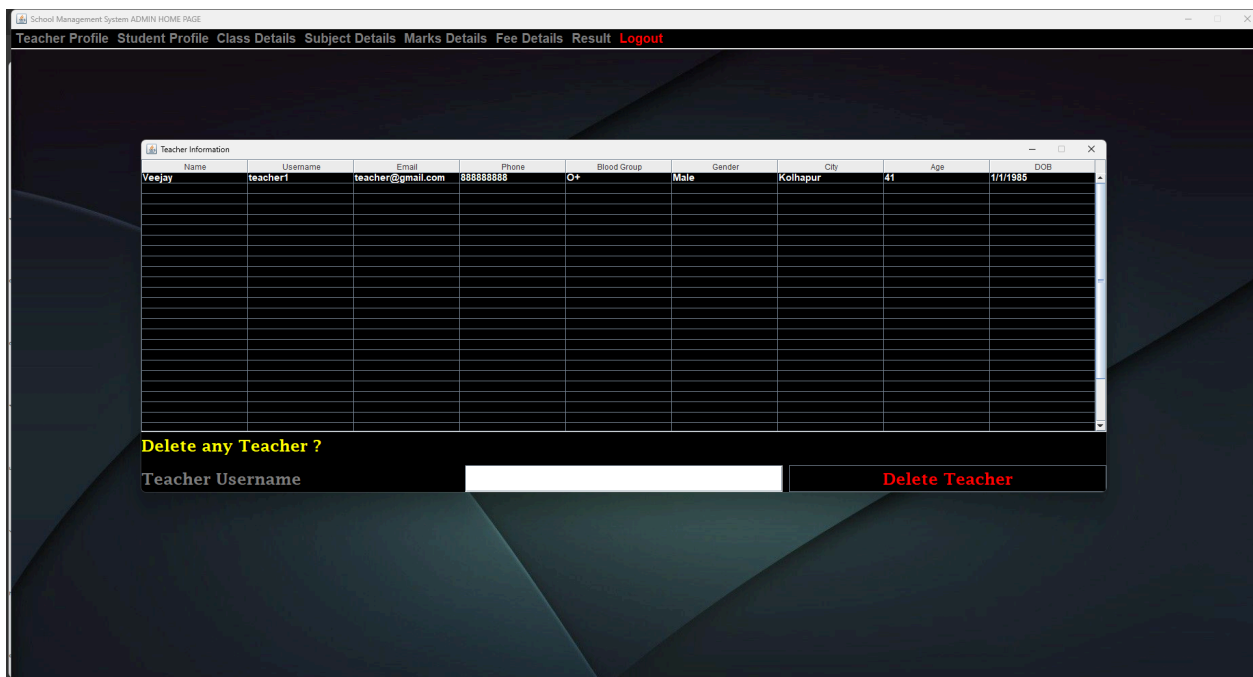


Image: Admin view of ViewTeacherDetails, After Delete

```
mysql> select * from teacher;
```

tec_id	name	username	password	email	father_name	phone	blood_group	gender	city	age	teaching_exp	dob
22695	Veejay	teacher1	teacher123	teacher@gmail.com	Beejay	888888888	O+	Male	Kolhapur	41	15	1/1/1985

1 row in set (0.00 sec)

Record in Database viewed from MySQL 8.0 Command Line Client, After Delete

```

97      }
98      catch(Exception ex)
99      {
100          ex.printStackTrace();
101      }
102  }
103
104  public void actionPerformed (ActionEvent e)
105  {
106      if(e.getSource() == b1)
107      {
108          String username=b1.getText();
109          try
110          {
111              ConnectionClass obj=new ConnectionClass();
112              String q="delete from teacher where username='"+username+"'";
113              int res=obj.stm.executeUpdate( string: q);
114              if(res==1)
115              {
116                  JOptionPane.showMessageDialog(parentComponent: null, message: "Teacher record is deleted !");
117                  setVisible( b: false);
118              }
119              else
120              {
121                  JOptionPane.showMessageDialog(parentComponent: null, message: "Teacher record does not match !");
122              }
123          }
124          catch(Exception ex)
125          {
126              ex.printStackTrace();
127          }
128      }
129  }
130
131  // public static void main(String[] args)
132  // {
133  //     new ViewTeacherDetails("account", "username").setVisible(true);
134  // }
135  }
136

```

Code from ViewTeacherDetails.java

Try & Catch Block

In Java, the try and catch blocks is a special feature that helps to manage errors that happen while a program is running. An alternative to crashing when something goes wrong, this allows the program to catch the error and respond to it properly. This way, you can prevent unexpected failures and keep the program running smoothly. I have used these blocks to handle in each and every place in my developed software to handle runtime errors effectively and efficiently

```
try
{
    ConnectionClass obj=new ConnectionClass();
    String class_name=ch1.getSelectedItemAt();
    String username=ch2.getSelectedItemAt();

    String q="select * from marks where class_name='"+class_name+"' and username='"+username+"'";
    ResultSet restl=obj.stm.executeQuery( string: q);

    while(restl.next())
    {
        ta.append("\n\n Student Name : " +restl.getString( string: "name"));
        ta.append("\n Div " +restl.getString( string: "term"));
        ta.append( str: "\n =====\n");
        ta.append("\n"+restl.getString( string: "subject_name")+ " : "+restl.getString( string: "marks"));

        ta.append( str: "\n =====\n");

        total=Integer.parseInt( s: restl.getString( string: "marks"))+total;
        float div = (total * 100f) / 500f;
        ta.append("\nTotal : " +total);
        ta.append("\nDiv : " +div+"%");
    }
}
catch(Exception ex)
{
    ex.printStackTrace();
}
```

Code from the ShowResults.java file

Bibliography

- “Normal Framework.” *Normal.dev*, 2020, docs2.normal.dev/. Accessed 1 Sept. 2025.

Word Count Criteria C: 1402 words